



Universidade Federal de Viçosa – Campus UFV-Florestal
Ciência da Computação – Projeto e Análise de Algoritmos
Professor: Daniel Mendes Barbosa
Aluno: Gabriel S. Ferreira de Pádua
Trabalho Prático 0



Introdução.....	3
Organização.....	4
docs:.....	4
output:.....	4
src:.....	4
makefile:.....	4
Desenvolvimento e Dificuldades.....	5
Execução.....	6
Usabilidade.....	7
Área de Teste.....	8
Algoritmo de Montagem.....	9
Aleatoriedade.....	10
Posição na Matriz (x,y no quadro).....	10
Quantidade de imagens (Aleatórias).....	10
Obra criativa - Lorax.....	13
Resultados.....	15
Conclusão.....	16
Referências.....	16

Introdução

Este Trabalho prático trata-se de um simulador de quadro de arte, denominado **SMART**, no qual o usuário pode escolher entre símbolos predeterminados juntamente com a quantidade que ele desejar, e assim criar “obras de arte” que serão impressas dentro deste quadro.

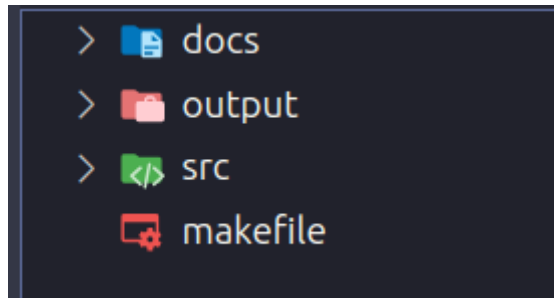
É incluso também uma mini historinha interativa, no qual dependendo de suas escolhas, uma 2 tipos de obras de artes podem ser criadas. Esta visa a criatividade do desenvolvedor, no qual foi escolhido por mim algo relacionado ao clima e as recentes mudanças dele, que se continuarem assim podem chegar a níveis drásticos, com isso propus uma obra dinâmica relacionada ao filme : “O LORAX”.



Filme: O lorax

Organização

O programa encontra-se seguindo uma organização no qual onde cada trecho ou subparte sua seja de fácil entendimento e esteja dentro de pastas devidamente nomeadas:



Pasta principal

docs:

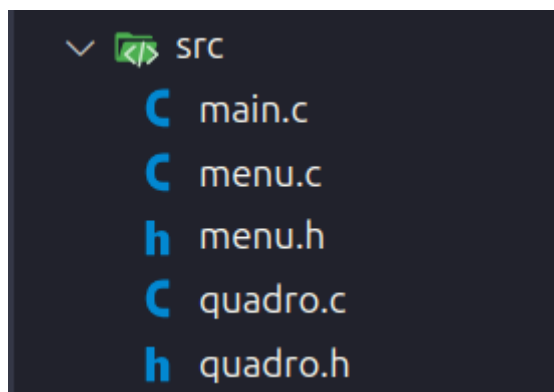
Estará disposta toda a documentação, tanto a de especificação do trabalho quanto a que vós leem.

output:

Pasta no qual o programa mandará o seu arquivo executável (esta pasta é essencial para o funcionamento correto do programa)

src:

Pasta que contém o código fonte, sendo os TADs e o programa central, a “main.c” propriamente



src/

makefile:

Estão os comando make, que automatizam os códigos necessários para compilar e executar o programa

Desenvolvimento e Dificuldades

O desenvolvimento, assim como qualquer outro programa, por mais simples que seja apresentou dificuldades em algumas partes de seu processo.

Para o algoritmo, usei conhecimentos de funcionamento de assentamentos de estruturas presente em jogos como Age of Mythology/Empires ou Clash of Clans, no qual dado um item que será acrescentado em uma matriz(mapa), se alguma parte deste elemento estiver encostando em outra, ou estiver em cima ou fora do mapa (limite da matriz) ele não pode ser assentado, garantindo assim que nenhum item sobrescreva a outros.

Na parte do algoritmo reservada ao desenvolvimento criativo, a dificuldade ficou restrita em identificar os lugares específicos e em alterar as cores dos desenhos, visto que após a colocar item na matriz, estes apresentam apenas abstrações na memória, no qual só recebem cor ao serem mostrados no terminal. Portanto deveria ser alterada a cor dependendo do caracter durante a amostragem no terminal e retornada a cor original após os processos de coloração dos objetos.



exemplo assentamento ou estruturas colocadas no mapa (imagem meramente ilustrativa).

Execução

O programa foi desenvolvido em uma máquina Linux - Ubuntu , e usado o compilador GCC.

Caso não tenha o GCC basta usar o seguinte comando no terminal:

```
sudo apt-get install gcc
```

ou no caso do windows instalar o MinGW.

A execução do programa é bem simplificada graças ao makefile. Basta abrir o diretório da pasta principal com o terminal e digitar os seguintes comandos entre as aspas:

- **“make”**: irá apenas compilar o programa
- **“make compile”**: irá também compilar o programa
- **“make run”**: irá executar o programa anteriormente compilado
- **“make all”**: irá compilar e executar o programa

Usabilidade

A usabilidade do programa é de fácil entendimento, com o usuário sendo apresentado a um menu de opções, no qual ele deve escolher e seguir com as entradas solicitadas.

```
Escolha o tipo de figura que deseja para criar sua obra:  
  1 - Asteriscos Simples  
  2 - Simbolo de soma com asteriscos  
  3 - Letra X com asteriscos  
  4 - Gerar Figura aleatoria com todos  
  5 - Area de testes:  
  6 - Conversa com LORAX  
  7 - Sair  
Digite a opção desejada: █
```

Após a conclusão de qualquer opção, o usuário será encaminhado ao início novamente para caso deseje continuar, caso contrário bastará escolher a opção sair para encerrar o programa.

Área de Teste

Foi desenvolvida uma área de teste para o professor, que diferencia todos os caracteres das obras de arte para melhor identificação.

A opção 5 do menu principal leva o usuário até o menu de testes

```
Escolha o teste que deseja realizar:
  1 - Teste asterisco simples
  2 - Teste soma com asteriscos
  3 - Teste x com asteriscos
  4 - Teste figuras aleatorias
  5 - Sair
Digite a opção desejada de teste: 4
Quantas figuras deseja criar? 20
```

menu de teste



saída por terminal

Algoritmo de Montagem

Um algoritmo genérico foi implementado, no qual a sua base de pensamento serviu para a montagem de todas as obras e desenhos que as formam, sendo esse algoritmo:

1. determinar um ponto na matriz (X,Y) - será o centro da figura
2. verificar se os espaços na matriz que formariam a figura contém o caractere <espaço>.
3. se, e somente se existir caractere <espaço> para toda posição em torno da posição central que formará o desenho, trocar <espaço> por outro caractere que forme o desenho

```
void asteriscoSimples (char matriz[20][80],int numeroDeAsteriscos){  
  
    int condicao = 0;  
  
    srand(time(NULL));  
    for (int i = 0; i < numeroDeAsteriscos; i++) {  
        int numeroX = numeroAleatorioLinha();  
        int numeroY = numeroAleatorioColuna();  
  
        while (condicao == 0) {  
            // Verifica se as coordenadas ao redor de (numeroX, numeroY) estão dentro da matriz  
            if (numeroX > 0 && numeroX < 19 && numeroY > 0 && numeroY < 79) {  
                // Verifica se todas as posições necessárias estão vazias  
                if (matriz[numeroX][numeroY] == ' ' ) {  
                    condicao = 1;  
                }  
            }  
            else {  
                // Gera novos números aleatórios se não estiver vazio  
                numeroX = numeroAleatorioLinha();  
                numeroY = numeroAleatorioColuna();  
            }  
        }  
        else {  
            // Gera novos números aleatórios se as coordenadas estiverem fora dos limites  
            numeroX = numeroAleatorioLinha();  
            numeroY = numeroAleatorioColuna();  
        }  
    }  
  
    // Desenha os asteriscos se a condição for atendida  
    if (condicao == 1) {  
        matriz[numeroX][numeroY] = '*';  
    }  
  
    // Reinicia a condicao para a próxima iteração  
    condicao = 0;  
}  
}
```

função que pode ilustrar o algoritmo

Aleatoriedade

Posição na Matriz (x,y no quadro)

o uso da aleatoriedade foi através de uma biblioteca nativa do C, a <time.h> que possibilitou que eu usasse o tempo cronológico e atual em segundos como semente e posteriormente fazer o mod e adicionar 1, para que eu pudesse conseguir os números no intervalo que eu desejasse.

```
void asteriscoSimples (char matriz[20][80],int numeroDeAsteriscos){  
    int condicao = 0;  
  
    srand(time(NULL));  
>    for (int i = 0; i < numeroDeAsteriscos; i++) {...
```

srand(time(null)) - usando o tempo do computador para gerar um número pseudo aleatório, visto que segue o tempo cronológico desde 1970.

```
int numeroAleatorioLinha(){  
    int numeroI = 1 + (rand() % 18);  
    return numeroI;  
}  
  
int numeroAleatorioColuna(){  
  
    int nuemroJ = 1 + (rand() % 78);  
    return nuemroJ;  
}
```

números aleatórios - posições de pares ordenados (X,Y) dentro da matriz (quadro)

Quantidade de imagens (Aleatórias)

Para o algoritmo que de quantidade de figuras discreto mas aleatórias entre si e de quantidade aleatórias de cada tipo houveram dois tipo de pensamentos implementado por mim, onde primeiramente podem parecer iguais em probabilidade, mas se analisarmos estatisticamente vemos que as chances são distintas e portanto foram usadas em diferentes contextos, o primeiro uso nas opções especificadas no trabalho, onde a chance de todas deviam ser igual de aparecer, e o segundo uso na minha ideia criativa, onde normalmente

em uma floresta, há a chance muito maior de terem árvores “verdes normais” e poucas árvores de outras cores como ipês ou com folhagem diferentes de verde

Primeiro uso: com o while, onde o número de figuras que o usuário pediu ia decremento, e cada iteração era sorteado um número de 1 a 3 ({1,2,3}) onde cada número representava um tipo de figura a ser anexada e exibida no quadro.

- Probabilidade de ambas eram de 33,33%.

```
void figurasAleatorias(char matriz[20][80], int numeroDeFiguras){  
  
    srand(time(NULL));  
  
    int figSimples = 0;  
    int figSoma = 0;  
    int figX = 0;  
  
    while (numeroDeFiguras > 0)  
    {  
        int daVez = 1 + (rand() % 3);  
  
        //faz obra simples  
        if (daVez == 1)  
        {  
            asteriscoSimples(matriz, 1);  
            figSimples++;  
        }  
        //faz obra com soma  
        else if (daVez == 2)  
        {  
            somaComAsteriscos(matriz, 1);  
            figSoma++;  
        }  
        //faz obra com X  
        else if (daVez == 3)  
        {  
            xComAsteriscos(matriz, 1);  
            figX++;  
        }  
        numeroDeFiguras--;  
    }  
  
    //printf("\tasteriscos simples: %d\n\tsimbolo de soma: %d\n\tsi  
}
```

Segundo uso: usei uma conta que a partir do número de figuras que o usuário solicitasse seria salvo em 3 variáveis o valor mutável/ indeterminado que juntos se completaria e seriam igual ao valor do usuário. Inicialmente achei que ambas as variáveis tinham o valor de 33,33% de chance de aparecer, mas logo vi que não já que a primeira tinha valor de:

- Probabilidade da primeira (P1): 1% a 100% de chance.
- Probabilidade da segunda (P2): 100% - P1.
- Probabilidade da terceira: (P3): 100% - (P1 + P2).

Embora isso não tire a chance ainda das outras árvores aparecem em muito mais quantidade.

```
void florestaSalva(char matriz[20][80],int numeroDeArvores){
    srand(time(NULL));

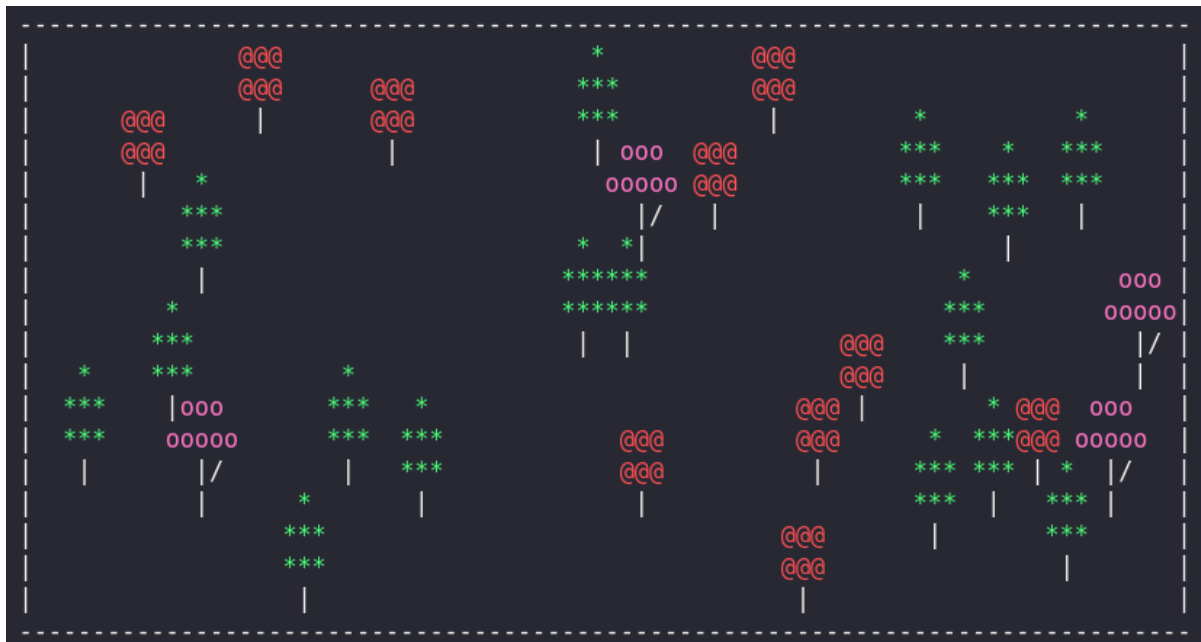
    int arvVerde = 0;
    int arvVermelha = 0;
    int arvRoxa = 0;

    ✨ arvVerde = 1 + (rand() % numeroDeArvores);
    arvVermelha = rand() % (numeroDeArvores - arvVerde);
    arvRoxa = numeroDeArvores - arvVerde - arvVermelha;

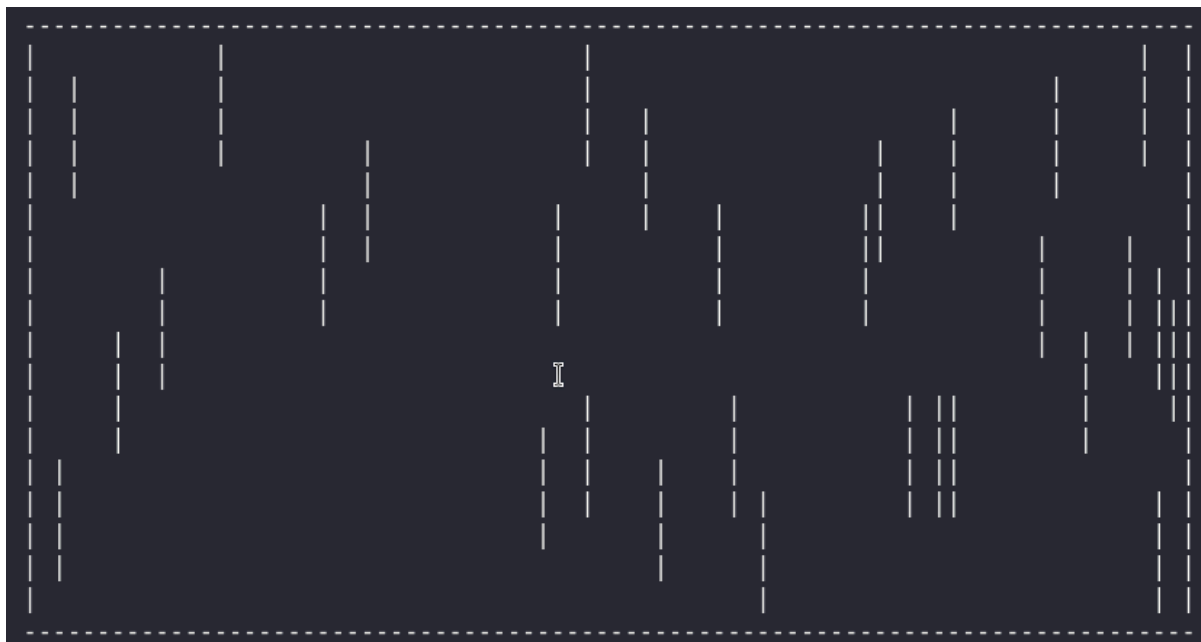
    arvoreCheia(matriz,arvVerde);
    arvoreDeMaca(matriz,arvVermelha);
    arvore2galhos(matriz,arvRoxa);
}
```

Obra criativa - Lorax

Artes Conversa com Lorax - Trata-se de uma pequena historinha interativa , no qual Lorax se apresentará e conversa com o usuário, no qual dependendo de suas escolhas pode se ver em um mundo com a natureza destruída por sua ganância, referenciando ao filme, ou em um mundo totalmente belo e com a natureza intacta.



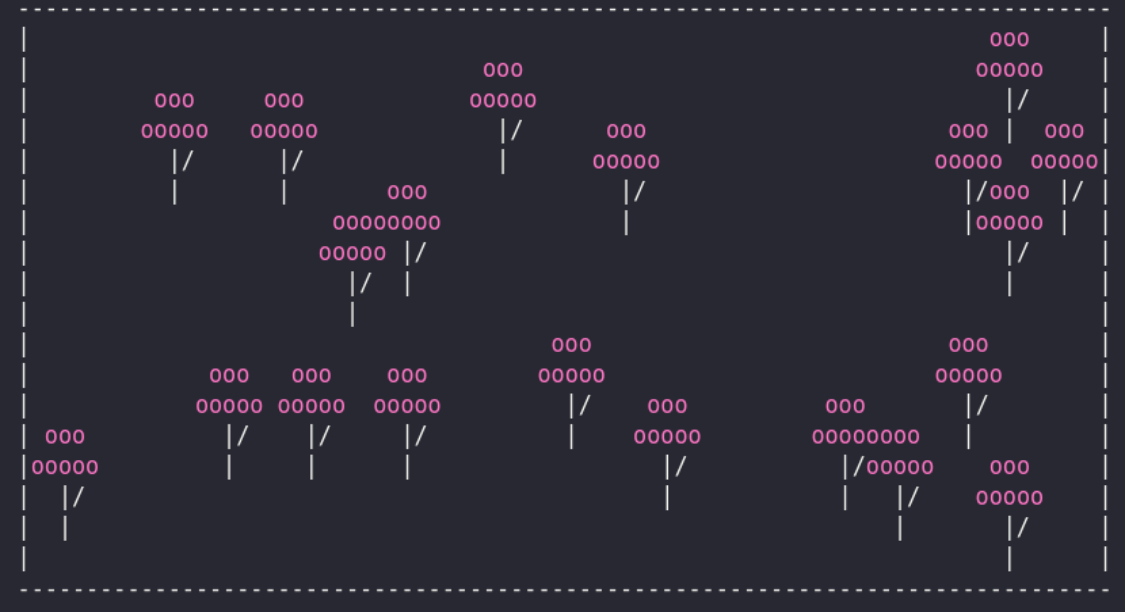
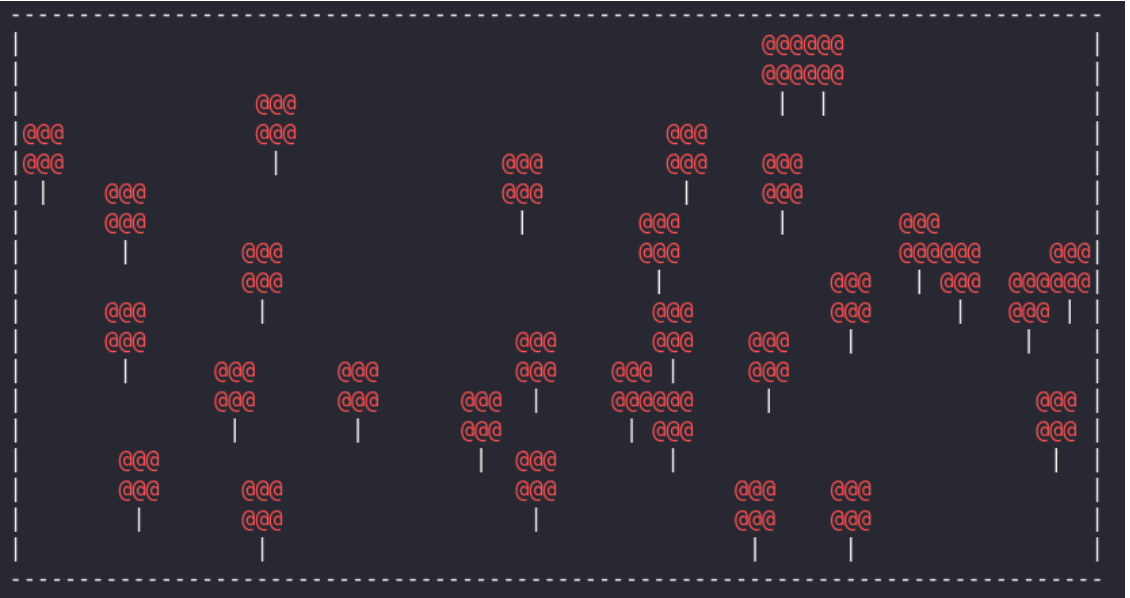
Floresta cheia de vida - Escolha sem Ganância



Floresta morta - árvores sem vida - Escolha com Ganância

Amostragem das árvores:

Trata-se apenas do algoritmo manipulado, para assim mostrar os tipos de árvores presentes que o usuário pode se deparar



Resultados

Todos os resultados foram satisfatórios e condizentes com o planejado. Abaixo, mostro algumas saídas do programa.

Artes Especificadas - 20 objetos para melhor compreensão:



Conclusão

Os resultados foram alcançados, houve um esclarecimento sobre aleatoriedade ou pseudo aleatoriedade. Um brainstorm tomou conta da minha cabeça. Coisas e utilidades que são possíveis ou até mesmo já vi em game e imaginei como eram feitas agora estão claras. Certamente esses conceitos poderiam ser reaproveitados futuramente

Referências

<https://www.vivaolinux.com.br/artigo/Aleatoriedade-em-C>

<https://marmsx.msxall.com/cursos/c/c22.html>